Sixteen codes for fifteen values: a lossless synonymous codon in MXFP4, and gpt-oss-120b served from disk on a 16 GB laptop

Stone

29 October 2026

Abstract

MXFP4, the 4-bit format in which gpt-oss publishes its expert weights, has sixteen codes for fifteen values: +0 and −0 are two codes of the same zero. The quantiser of gpt-oss keeps the sign of values rounded to zero; over the 19 110 297 600 expert weights of gpt-oss-20b, that sign is a fair coin (entropy 0.9999999967 bit), and no computation of the engines considered depends on it. We prove that folding −0 into +0 before entropy coding saves exactly $\mathbb{P}(V = 0)\, H(S \mid V = 0)$ bit per weight, twice the share of −0 codes when the sign is a fair coin, and we check it on all 180 expert tensors of gpt-oss-20b and gpt-oss-120b to within 1.7×10^{-4} bit. Carried into version 1.2 of the `.trc` format of the troncons engine by its coding tables alone, decoder and kernels unchanged, the fold removes 1 800 486 912 bytes (1.80 GB) from gpt-oss-120b, 0.12562 bit per expert weight (3.26 % of its bits), and the engine's logits stay identical to the bit over 5 120 steps on both models. troncons serves gpt-oss-120b (63.4 GB) on a laptop with 16 GB of RAM and an 8 GB RTX 4060, with a host working set of 384 MB and a GPU footprint set by the card, not by the model (about 6.3 GB of its 8 GB). On this laptop, in an ABBA series, the v1.2 file generates 2.890 tokens/s against 1.988 for v1.0 (×1.454; an independent judge: ×1.442), whereas its 3.3 % fewer bytes would allow at most +3.4 %: a crossed experiment on gpt-oss-20b, at the disk stage, shows that the gain follows the pattern of read requests that the format imposes on this disk. It is a result of this machine, not a portable one. We also report what did not work, each against its declared threshold, and the method: thresholds committed before measuring, controls that can fail, independent judges.

1 Introduction

The project behind this paper set itself a concrete goal: run, on an ordinary laptop, a language model much larger than all of the laptop's memory, *exactly* (to the bit, the logits that the same engine computes from the uncompressed weights), by storing its weights compressed on disk and decoding only what a computation needs, at the moment it needs it. The machine has 16 GB of RAM, an RTX 4060 Laptop GPU with 8 GB, and a 1 TB NVMe disk encrypted by BitLocker. The model is gpt-oss-120b [1], a mixture of experts of 63.4 GB in its native MXFP4 format: four times the RAM, eight times the GPU memory.

Three technical volumes report the work in full, each in English and French. Volume I [2] measured the machine and showed that, on it, everything comes down to the number of bits read from disk per weight. Volume II [3] built the engine, troncons, and judged its first milestone (J1) against twelve criteria declared before the first line of code. Volume III [4] searched for a way to read fewer bits and built what held. This short paper is their entry door. It reports one small, exact idea and what it did on this machine, and it says plainly what did not work.

The idea is a synonym. In the genetic code, several codons give the same amino acid [5]; in MXFP4, two codes give the same zero. When the sign of a zero carries no information, which is the case in gpt-oss, storing it costs almost one bit per zero, and about one expert weight in eight is a zero. Removing it is lossless as long as every reader of the codes reads both codes as zero, and that is a property one can prove on the real kernel. The spare code of zero is known, and other formats give it another value (Section 2); we do not, because we keep the model as it was published, every weight with the same value, and only stop paying for a sign that changes no value.

Contributions.

- A measurement, tensor by tensor on the 180 expert tensors of two published models, that the sign of a zero is a fair coin, and the exact, elementary statement of what folding the synonym saves, $H(C) = H(V) + \mathbb{P}(V = 0)\, H(S \mid V = 0)$ (Sections 3.2 and 4.1).
- A proof that the engine's MXFP4 matrix–vector kernel returns the same bits on both codes, and a lemma showing that the fold is carried into a format through its coding tables, with no change to the decoder (Sections 3.2 and 3.3).
- The `.trc` v1.2 files of gpt-oss-20b and gpt-oss-120b, checked expert by expert in value, 1.80 GB smaller on the 120b, with logits identical to the bit (Sections 4.2 and 4.3).
- Milestone J1 redone on gpt-oss-120b in ABBA series, and an attribution of its ×1.45 to the read pattern of the format on this disk, by a crossed experiment of an independent judge on gpt-oss-20b, at the disk stage (Sections 4.4 and 4.5).
- Negative results with their declared thresholds, and the measurement discipline that produced all of the above (Sections 3.4 and 5).

Labels follow the volumes: [meas.] measured on this machine, tool and log committed; [calc.] computed from committed measurements; [lit.] literature, not reproduced. Paths such as `genome/06-codon-v1-2.md` are relative to the troncons repository (Section 7).

2 Related work

Serving models larger than memory. FlexGen offloads weights and cache across GPU, CPU and disk to maximise the throughput of batched generation with a single GPU [6]. LLM in a Flash reads weights from flash storage on demand, with windowing and row–column bundling to enlarge reads [7]. PowerInfer splits neurons into a hot set kept on the GPU and a cold set computed on the CPU [8]. For mixtures of experts, Eliseev and Mazur [9] keep an LRU cache of experts on the GPU and load others speculatively; MoE-Infinity designs an expert cache for personal machines [10]; Fiddler computes cold experts on the CPU instead of moving them [11]. llama.cpp offloads MoE layers between host and GPU and is the baseline of Volume II [12]. troncons differs in its contract: weights stay *losslessly entropy-coded* on disk and are decoded on the GPU at the moment of use, and the logits are, to the bit, those that the engine computes from the raw GGUF weights; the expert cache and its theory (Belady's optimality [13], the bounds of LRU) are in Volume II.

Lossless compression of weights. ZipNN [14], DFloat11 [15] and Huff-LLM [16] compress weights without loss, mostly by entropy coding the redundant part of floating-point representations such as BF16 exponents. Lossy sub-bit compression of very large MoEs also exists [17]. We work one level lower, inside a format that is already four bits: MXFP4 [18, 19] stores blocks of 32 E2M1 codes sharing an E8M0 scale. Its redundancy that we exploit is not statistical skew but a *synonym*: two codes, one value, by the signed zero of IEEE 754 [20]. That redundancy is known. BitMoD replaces the redundant zero of FP3 and FP4 variants by a special value [21]; the code recycling of NxFP remaps the −0 code of sign-magnitude formats to half the smallest quantisation level, and applies it to MXFP4 [22]; RaZeR remaps the redundant zero of NVFP4 to further quantisation values [23]. All three are new, lossy formats into which a model is quantised again, and none measures how often −0 occurs in a published model. Lossless coding has been extended to FP8 and FP4 [24], but for FP4 it compresses only the block scales and stores the 4-bit values uncompressed, reporting that their streams appear statistically uniform; in gpt-oss-20b, the entropy of the codes is 3.8638 bits per weight, not 4 (Section 3.2), and our coder conditions them on the scale class. We know of no work that measures, in a model already published in MXFP4, whether the sign of a zero carries information, and removes it without loss, with a proof that the kernel returns the same bits. The entropy coder is interleaved rANS [25, 26]; the information-theoretic tools are standard [27]; the quality judge is the Kullback–Leibler divergence [28] against a double-precision reference.

Ideas that we tested and that failed. The router of an MoE layer partitions its input space into polyhedral regions, as a tropical polynomial does [29]; weight matrices of trained networks often show heavy-tailed

spectra [30], which one tests against a power law by the method of Clauset et al. [31]. Section 5 reports what these readings gave on gpt-oss.

3 Method

3.1 The machine, the model and the engine

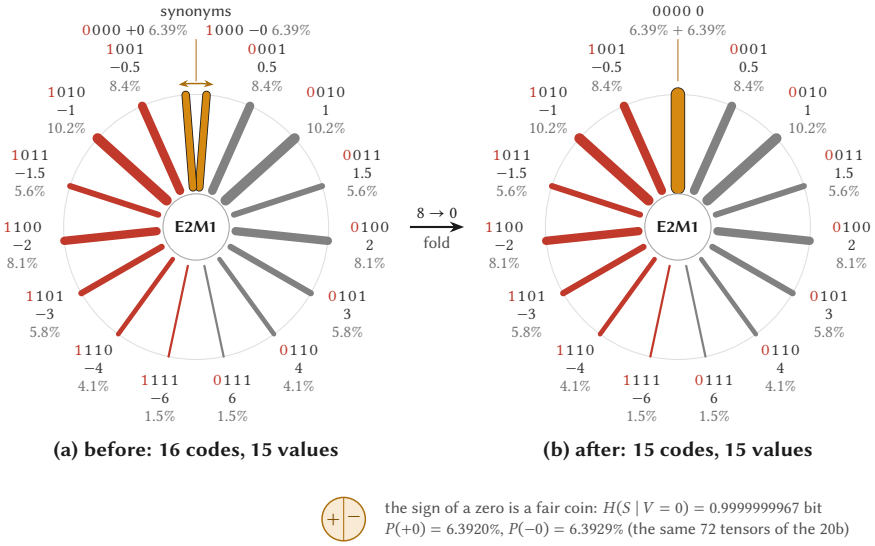
The machine [meas.]. Windows 11, an i7-13620H (16 threads), 16 GB of RAM (15.6 usable), an RTX 4060 Laptop GPU with 8 188 MiB on a PCIe 4.0 ×8 link, a 1 TB Samsung NVMe disk behind Intel RST VMD, under BitLocker. Volume I measured 2.82 GB/s from the disk (`fondations/journaux/01-machine/debits-mesures.json`).

The model [lit.] [meas.]. gpt-oss-120b has 36 layers of 128 experts, of which the router of each layer chooses four per token; its GGUF file is 63 387 346 208 bytes, its 4 608 experts 61.07 GB, and every token also reads 1.69 GB of attention, routers and output head (`mesures/gpt-oss.md`). Its development twin, gpt-oss-20b (24 layers of 32 experts, 12 109 566 624 bytes), is already larger than the GPU.

The engine and its byte path. troncons is a Rust workspace with CUDA kernels loaded through the driver API alone (Volume II, chapters 1 to 6). One `.trc` file per model, aligned on 4 096 bytes, holds one *chunk* per expert: the expert's three MXFP4 matrices, entropy-coded by interleaved rANS with 32 lanes, each lane knowing from its position which table to use (one table per scale class κ), plus a directory with the offset, length and fingerprint of every chunk. The resident weights (attention, routers, head) live in VRAM; experts live in a VRAM cache of fixed slots. When a router asks for an expert that is not in a slot (a miss), its chunk travels four stages:

1. **disk to host**: one unbuffered read (`FILE_FLAG_NO_BUFFERING`) per chunk, issued without waiting and harvested from an I/O completion port [32], straight into pinned host memory;
2. **host to GPU**: a DMA from that same pinned page over PCIe; the GPU has a single copy engine under WDDM, a fact Volume II measured;
3. **decoding**: a CUDA rANS decoder turns the chunk into an expert slot, proved to return exactly the scalar decoding;
4. **compute**: an MXFP4 matrix–vector kernel (GEMV) reads the slot; every reduction follows a fixed tree, so two runs on the same bytes return the same bits.

A production guard compares the fingerprint of every chunk read with the directory and turns any corruption into a named fatal error. Volume II's results on gpt-oss-120b, with the v1.0 format: a peak host working set of 384 MB (384.19 MB, pinned memory included, against 380.50 MB for the 20b, which is five times smaller), where llama.cpp holds 10.93 GB of host memory in its best published configuration, while troncons's private committed memory, 6.93 GB (llama.cpp: 6.81 GB), is, as Volume II reads it (published there, not judged), of the order of its GPU allocations and pinned memory, reserved against the commit limit of Windows without occupying physical RAM; both engines also use the GPU, troncons about 6.3 GB of its 8 GB (6 317 303 600 bytes), a share set by the card, not by the model, and nvidia-smi reads the whole card, other processes included, at a peak of 6 574 MiB under troncons and of 6 026 and 6 027 MiB under llama.cpp in that configuration (independent trial of Volume II, `mesures/moteur/epreuve-J1/csv/s1-2026-10-16-passes.csv`, which also holds the committed memory), so that the large gap between the two engines lies in the host working set; logits identical to the bit between the coded and the raw file over 5 120 steps, by the same engine; 1.965 tokens/s with the guard, ×2.22 the best llama.cpp configuration measured (0.884 tokens/s; two sessions of the same morning, Volume II); and, on window 0 (255 positions) with an f16 KV cache, a KL divergence to an f64 reference of 0.00308, against 0.0607 for llama.cpp [meas.] (Volume II, chapter 7; `mesures/moteur/O/`, `mesures/moteur/epreuve-J1/`). Volume II traces most of llama.cpp's gap, on the 20b, to its rounding of the YaRN ramp [33] rather than to kernel precision; with an f32 KV cache, troncons's KL is −0.00048, at the resolution floor of the logit files themselves (`.kld`, $\log q$ on 16 bits). Host memory no longer follows the size of the model.



(a) before: 16 codes, 15 values **(b) after: 15 codes, 15 values**

Figure 1: The fold $8 \to 0$ removes one of the two codes of zero. The fifteen values of E2M1 at equal steps around a circle (0 at the top; values of the OCP standard, which llama.cpp doubles). The width of a ray grows with the measured share of its code in the 72 expert tensors of gpt-oss-20b (19 110 297 600 weights), affinely (0.4 pt + 44 pt × share), so the zero ray thickens by ×1.88, not ×2. Before: `0000` (+0, 6.3920 %) and `1000` (−0, 6.3929 %) for one value; after, one ray carries both. The sign of a zero is a fair coin. Analogy, not proof: GGU, GGC, GGA and GGG all give glycine. [meas.] Data: `mesures/genome/predictif-elagage/histo-toutes-gptoss20b.npz`, `mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`.

3.2 The synonymous codon of zero

E2M1 has one sign bit and three magnitude bits. llama.cpp and troncons double its magnitudes into the integers $K = (0, 1, 2, 3, 4, 6, 8, 12)$ and move the factor 2 into the scale. For a code $c \in \{0, \dots, 15\}$ with bits $c_3 c_2 c_1 c_0$,

$$v(c) = (-1)^{c_3} \, K[c \,\&\, 7]. \tag{1}$$

The map v is injective except on $\{0, 8\}$: $v(0) = +0$ and $v(8) = -0$, and $-0 = +0$ in IEEE 754. Sixteen codes, fifteen values (Figure 1).

The coin [meas.]. The quantiser of gpt-oss keeps the sign of a value rounded to zero. Over the 72 expert tensors of gpt-oss-20b, the code +0 has the share 6.3920 % and −0 the share 6.3929 %: the sign of a zero has an entropy of 0.9999999967 bit. The entropy of the codes is 3.8638 bits per weight, that of the values 3.7360; the difference, 0.1278 bit, is what a sign that changes no value costs (`mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`).

Proposition 1 (Entropy of a code with a synonym)**.** *Let C be the code of a weight, $V = v(C)$ its value and $S = C_3$ its sign bit. Then $H(C) = H(V) + \mathbb{P}(V = 0)\, H(S \mid V = 0)$.*

Proof. V is a function of C, so $H(C, V) = H(C)$, and by the chain rule $H(C, V) = H(V) + H(C \mid V)$. Given $V = v \neq 0$, C is determined, since v is injective away from 0. Given $V = 0$, $C \in \{0, 8\}$ is determined by S. Hence $H(C \mid V) = \mathbb{P}(V = 0)\, H(S \mid V = 0)$. □

Corollary 2 (The gain of the fold)**.** *Replacing C by $C' = \gamma(C)$, with $\gamma(8) = 0$ and γ the identity elsewhere, makes C' a bijection of V, so the entropy falls by $\mathbb{P}(V = 0)\, H(S \mid V = 0)$ bit per weight. If the sign of a zero is a fair coin, the −0 codes are half of the zeros and*

$$\mathit{gain} = \mathbb{P}(V = 0) = 2 \times (\mathit{share\ of} -0\ \mathit{codes}). \tag{2}$$

The same holds conditionally on any context X the decoder knows (scale class, neighbouring byte): replace $H(\cdot)$ by $H(\cdot \mid X)$ throughout. If the context predicted the sign, the gain would be smaller than $\mathbb{P}(V = 0)$.

Remark 3 (Bytes). The coder works on bytes, two codes at a time. The fold removes from the alphabet the bytes one of whose nibbles is 8: $16 + 16 - 1 = 31$ bytes, so $256 - 31 = 225$ remain [calc.]. Applied to the byte and its pair of values, the same proof gives $\mathbb{P}(V = 0)$ bit per weight when the signs of zeros are fair coins, independent of everything else.

The fold is lossless only if *every* reader of the codes gives code 8 the value 0. The project lists them (`genome/06-codon-v1-2.md` § 1): ggml reads the codes through an int8 table where entry 8 is 0 [lit.]; the GPU decoder copies bytes without reading values; the f64 reference reads the GGUF, not the `.trc`; and the only reader of values in the engine's kernels is the GEMV, where code 8 is −0.0f. The question is therefore that of the *bits* of a sum in which one term goes from −0 to +0.

Proposition 4 (The GEMV returns the same bits; Volume III, Proposition A1)**.** *The engine's MXFP4 GEMV (`moteur/crates/trc-noyaux/noyaux/gemv.cuh`, in its version of commit `4473050`, the one in the tree at `000cef9`) starts each accumulator at $s = +0$ and advances by $s \leftarrow \mathrm{fma}(v, x, s)$ in a fixed order, with v the doubled signed magnitude of the code. If the inputs x are finite, replacing code 8 by code 0 changes no bit of s at any step.*

Sketch. (0) Every finite binary32 is a multiple of 2^{-149} and v is an integer, so every exact sum $vx + s$ lies on the grid $2^{-149}\mathbb{Z}$; if it is not zero, its rounding is not zero. (1) The accumulator is never −0: it starts at +0; an exact zero sum rounds to +0 unless both terms are −0, which (by induction) s is not; a non-zero sum does not round to zero by (0). (2) For $v = \pm 0$, $vx = \pm 0$ exactly; if $s \neq 0$, $s' = s$; if $s = 0$ then $s = +0$ and $s' = +0$ whatever the sign of vx. (3) Other steps have bit-equal operands. By induction, s is bit-equal in both computations; the fixed-tree reduction then adds bit-equal partial sums, and the scale and bias are applied afterwards. □

The proof uses the fact that magnitudes are doubled to integers and that the scale stays outside the accumulation; a kernel that put the scale inside, or started a GEMM from −0, would leave it. The project therefore wrote an invariant into its architecture: every code reader reads code 8 as zero, and every new kernel that reads codes carries a control that compares an expert with its canonicalised copy. The same induction holds for the f64 reference under one hypothesis, stated in Volume III: no non-zero exact sum underflows to zero.

3.3 The `.trc` v1.2 format

The coder canonicalises the codes of each expert ($8 \to 0$) before counting symbols and building its rANS tables, so the 31 bytes that carry a nibble 8 receive frequency zero. Nothing else changes (Table 1).

Lemma 5 (The invariant of the tables suffices)**.** *If the table of a context gives $f[s] = 0$ to a symbol s, no stream, correct or corrupted, makes the decoder produce s.*

Proof. The decoder reads slot $i = x \bmod 4096$ of its state x and returns the symbol that carries it; by construction slot i carries the s with $c[s] \leq i < c[s] + f[s]$, c the cumulative frequencies. If $f[s] = 0$ the interval is empty and no slot carries s, whatever x. □

Two engineering consequences follow. The guarantee "no v1.2 stream produces a −0" is checked once, on the tables, at opening ($36 \times 3 \times 3 \times 31 = 10\,044$ reads for the 120b [calc.]), not per symbol. And *a v1.2 stream is a v1.0 stream whose effective alphabet has 225 bytes*: the GPU decoder and the GEMV have not had a single line changed. A synonym enters a format through its tables, not through its decoder. Because a v1.2 file is no longer bit-equal to the GGUF but equal in value, each directory entry keeps two fingerprints, the original one (provenance) and the canonical one (what the decoder must return), and the verifier compares values, not only bytes.

3.4 How the measurements were made

The numbers of this paper were produced under rules that the project applies to every step, and that matter as much as the numbers.

Table 1: What changes from `.trc` v1.0 to v1.2, and nothing else.

Where	Offset	v1.0	v1.2
header, minor version	10 (u16)	0	**2**
header, flags	12 (u32)	bits 0 to 2	+ **bit 3, canonical**
directory, each entry	56 (u64)	reserved, 0	fingerprint of the canonical expert
CODES_CTX tables	—	256 frequencies	built on canonical codes; frequency 0 for the 31 bytes that carry an 8, checked at opening

Source: `decisions/ARCHITECTURE.md` § 4.16; `genome/06-codon-v1-2.md` § 2. The reader refuses bit 3 under a minor version below 2, on a raw or two-level file, a canonical fingerprint without bit 3, and a canonical table that gives a slot to a byte carrying an 8; each refusal has its test and its break.

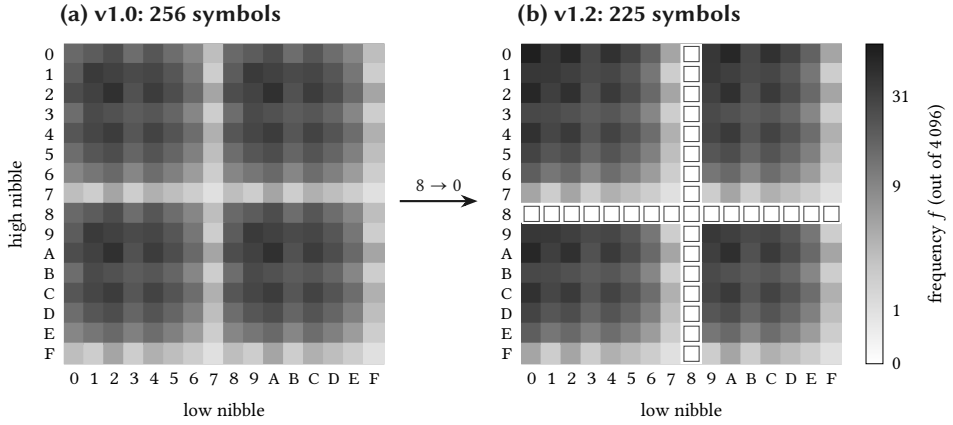


Figure 2: After the fold, row and column 8 go out: 225 bytes remain of 256. The CODES_CTX table ($\kappa = 1$) of gpt-oss-120b, layer 18, up matrix, as a 16×16 grid (high nibble in rows, low nibble in columns); grey level $\log_{10}(1 + f)$, f the frequency out of 4 096. Framed white cells: frequency zero, absent. Their mass moves onto row and column 0. Over all 540 CODES_CTX tables of both models, a second reader written from the specification alone finds a slot for a byte carrying an 8 in 540 v1.0 tables (13 % of the slots on average) and in none of the v1.2 tables. [meas.] Data: `mesures/genome/construction/codon/tables-octets.csv`, `tables-octets.json`.

- **Thresholds declared before measuring.** Each criterion and its threshold are written and committed before the first pass; the threshold derives from a named, measured quantity. The criteria of the v1.2 step (A-C1 logits, A-C2 size, A-C3 speed) are in the header of `outils/genome/construction/codon/pilote_codon.py`, commit `a50c380`, before any pass; the amendment of the calm session s2 is commit `71905dd`, before its passes.
- **Controls that can fail, and breaks.** Every measurement has a control whose answer is known in advance, and every new control is broken on purpose to check that the instrument falls. Examples: a coder run without the new option must write v1.0 byte for byte (same SHA-256 as the file of 15 October); a falsified median in one expert must make the logits differ (512 rows out of 512); a nibble 0 changed into 8 in a decoded expert must fail the byte checks and pass the value check.
- **ABBA series.** A cost is never based on one pass: fresh processes alternate A B B A A B B A, and the noise is the spread of a program against itself. A gain is announced only if $r - 1 > 2\sigma$, σ the relative standard deviation of the A passes; no regression is declared if $r \geq 1 - 2\sigma$. Passes run on mains power, under a lock on the GPU, and are discarded if another process loaded the machine (more than 5 % of a core or 8 MB read in 3 s).
- **Independent judges.** Each step is redone by a judge who wrote neither the decision nor the code, with their own tools: here, a decoder of the format written from its description alone, a GGUF reader, fresh text windows nobody had seen, a three-file speed session, and a disk reader of their own.
- **Strike-throughs.** When a measurement contradicts a published sentence, even our own, the sentence is struck through where it stands and what changed is written next to it. This paper carries one such correction (Section 4.5).

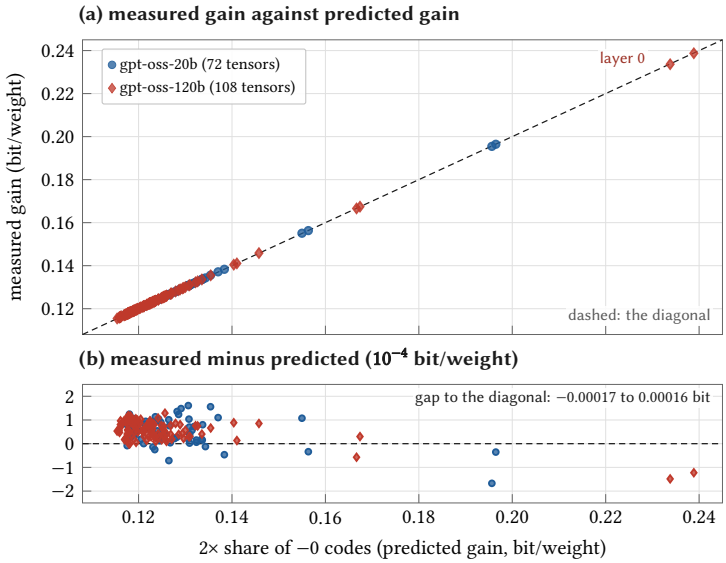


Figure 3: The 180 expert tensors of the two models sit on the diagonal that Corollary 2 predicts. (a) Measured gain of the fold (bits per expert weight, code streams of each tensor) against twice the share of −0 codes; the two points of layer 0 of the 120b are the richest in zeros. (b) Measured minus predicted, in 10^{-4} bit per weight: the context κ and the byte alphabet move no tensor beyond −0.00017 to +0.00016 bit. [meas.] [calc.] Data: `mesures/genome/construction/codon/tailles-par-tenseur.csv`.

4 Results

4.1 The gain, tensor by tensor

The coder folded 1 221 698 587 nibbles 8 on gpt-oss-20b (6.393 % of its expert weights) and 7 199 491 948 on gpt-oss-120b (6.279 %); the verifier, which canonicalises the slices of the GGUF by its own path, counts the same numbers to the unit [meas.] (`genome/06-codon-v1-2.md` § 5). Corollary 2 therefore predicts 2×0.0639288 = 0.12786 bit (20b) and 2 × 0.0627889 = 0.12558 bit (120b); the gain measured on the coded chunks is 0.12791 and 0.12562 bit [calc.]. On each of the 180 expert tensors (72 of the 20b, 108 of the 120b), the gap between the measured gain and twice the share of −0 codes lies between −0.00017 and +0.00016 bit, +0.00005 on average (Figure 3). This is stronger than it looks: had the scale class or the neighbouring byte, both known to the coder, predicted the sign of a zero, the measured gain would fall below $\mathbb{P}(V = 0)$. In every tensor, the sign of a zero is a fair coin that neither the scale class nor the other nibble of its byte predicts, and the rANS coder recovers its whole bit. The gain ranges from 0.115 bit (120b, layer 26, up) to 0.239 bit (120b, layer 0, up, where 11.9 % of the weights are −0).

4.2 Sizes on disk

The gain per expert weight is the difference of chunk bytes, in bits, over the number of expert weights; on the 120b [calc.],

$$\frac{(55\,416\,086\,528 - 53\,615\,599\,616) \times 8}{114\,661\,785\,600} = 0.12562 \text{ bit},$$

that is 0.12562/3.85456 = 3.26 % of the bits per expert weight, and 3.25 % of the chunk bytes (Table 2). The threshold A-C2, declared at 0.12 bit, holds on both models. If the disk alone limited speed, these bytes would allow 1/0.96751 − 1 = 3.4 % more tokens per second on the 120b [calc.].

4.3 Exactness

Experts, in value [meas.]. The verifier decodes every expert of both v1.2 files and checks eight properties, among them the decoded bytes against the GGUF canonicalised by a second path, the absence of any 8, and equality *in value* of every weight: 768 experts out of 768 and 4 608 out of 4 608 are exact, with zero faults in

Table 2: What the fold returns on the disk [meas.].

	gpt-oss-20b	gpt-oss-120b
expert weights	19 110 297 600	114 661 785 600
stream bits per weight, v1.0 → v1.2	3.85057 → 3.72266	3.85456 → 3.72892
chunk bytes, v1.0 → v1.2	9 226 493 952 → 8 920 932 352	55 416 086 528 → 53 615 599 616
gain per expert weight (A-C2: ≥ 0.12)	**0.12791 bit**	**0.12562 bit**
bytes saved	305 561 600	1 800 486 912
`.trc` file, v1.0 → v1.2	11 157 405 696 → 10 851 844 096	57 730 662 400 → 55 930 175 488
largest chunk, v1.0 → v1.2	12 144 640 → 11 755 520	12 251 136 → 11 878 400

Source: `mesures/genome/construction/codon/tailles.json`, `tailles-par-tenseur.csv` (tool `outils/genome/construction/codon/analyse_codon.py`). The stream-bits row counts the coded streams of the expert weights (medians, δ and codes; their difference, from the unrounded values, is 0.12563 bit on the 120b); the gain row counts chunk bytes. The judge's own reader of the format gives the four file sizes again to the byte (`genome/JUGEMENT-construction.md` § 2).

each check. Its two breaks find exactly the targeted expert; the second (a decoded 0 changed into 8: same value, wrong bytes) makes the byte checks fall and the value check hold, which is exactly the distinction v1.2 introduces (`genome/06-codon-v1-2.md` § 5). The judge decoded 96 randomly drawn experts with a decoder of their own, written from the description of the format alone: 96 out of 96 equal the GGUF expert to the bit up to the sign of zeros, and the nibbles that change between the v1.0 and v1.2 decodings are exactly the -0 codes of the GGUF.

The GEMV on the GPU [meas.]. A synthetic expert with the dimensions of gpt-oss (three 2880×2880 matrices) whose sign is drawn for every magnitude carries 2 486 216 nibbles 8; against its canonicalised copy, with arbitrary inputs (full mantissas, exponents from 2^{-10} to 2^{10}, zeros of both signs, non-exact sums), 8 640 **output rows, none differing at the bit**; a break (one code 0 set to 1 in the copy) makes row 0 differ.

Logits [meas.]. `troncons evaluer` computes the 64-bit fingerprint of the f32 logits at each of the 5 120 positions of ten windows of 512 tokens. v1.2 against v1.0: 5 120 out of 5 120 equal on gpt-oss-20b and on gpt-oss-120b, and the KL files equal byte for byte (`genome/06-codon-v1-2.md` § 7). On the 120b, v1.2 through the streamed prefill and the program of 17 October returns to the bit the logits that v1.0 returned through the step loop and the program of 15 October. The judge redid it on ten windows nobody had seen: 0 differing fingerprints out of 5 120 on each model, and a break of their own makes 512 out of 512 differ (`genome/JUGEMENT-construction.md` § 4). Proposition 4 holds for the real engine on new text.

4.4 Milestone J1 on gpt-oss-120b, v1.2 against v1.0

The milestone task is fixed since Volume I: a wikitext prompt, then 256 greedy tokens. Eight fresh processes alternate A B B A A B B A, A the original v1.0 file (written on 15 October), B the v1.2 file; the cache is fixed at 326 slots, the J1 number, so that the smaller largest chunk of v1.2 cannot free a slot and mix two effects. Session s2 ran on the night of 21 to 22 October on a calm machine; none of its passes was taken under load [meas.].

- **Tokens**: the 256 token identifiers are equal in the eight passes, and equal to the text of every 120b pass of 15 and 16 October; misses, hit rate and slots are identical. The format changes neither the routing nor the cache.
- **Bytes**, deterministic: $686\,756\,916 / 710\,118\,798 = 0.9671$, as predicted (0.967).
- **Speed**: medians 1.988 and 2.890 tokens/s, $r = 1.454$, $\sigma = 5.6\,\%$; paired ratios 1.450, 1.451, 1.444 and 1.626. At the declared threshold: no regression, and a **gain announced** ($0.454 > 2\sigma = 0.113$).
- **Memory**: -6.45 MB; 4 472 832 bytes of it are the 12 pinned arrival buffers sized on the largest chunk, $12 \times (12\,251\,136 - 11\,878\,400)$ [calc.]; about 2.0 MB are not explained.
- **Reproduction**: the judge put the three files of the 120b in one balanced session (A B R R B A A B R R B A, R a lossy companion), none of the 36 passes of the judge's two sessions (120b and 20b) under load:

Table 3: J1 on gpt-oss-120b, session s2: the eight passes, then the medians per token of the four passes of each file [meas.].

pass	1 A	2 B	3 B	4 A	5 A	6 B	7 B	8 A
tokens/s	1.981	2.873	2.892	1.994	2.006	2.897	2.887	1.776

median per token	A, v1.0	B, v1.2
generation, tokens/s	1.988	2.890
chunk bytes read	710 118 798	686 756 916
misses (cache hit rate)	59.08 (0.5897)	59.08 (0.5897)
time spent reading, s	0.391	0.259
read throughput seen by the engine, GB/s	1.81	2.65
decoding on the GPU, ms	62.1	42.7
host-to-GPU copy, ms	96.2	68.2
peak working set, MB	388.67	382.21

Source: `mesures/genome/construction/codon/j1-s2-passes.csv`, `j1-s2-verdicts.json`; medians by `memoire/article/outils/medianes_j1.py` (control: its token-rate medians equal those of the verdicts file).

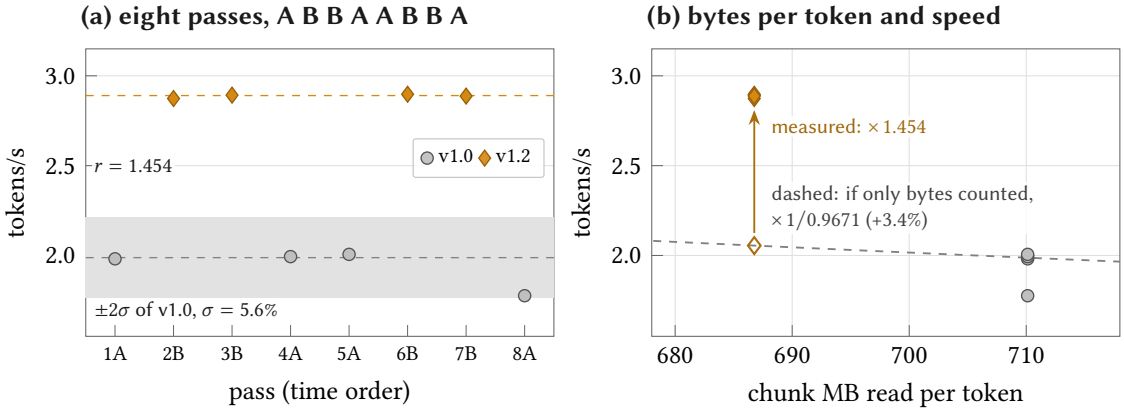


Figure 4: On the 120b, v1.2 runs ×1.454 **faster than v1.0, and its bytes would give it only** +3.4 % **at most.** Session s2, quiet machine, eight fresh processes, the same 256 tokens in all. (a) The passes in time order; dashed, the medians 1.988 (v1.0) and 2.890 (v1.2) tokens/s; grey band, $\pm 2\sigma$ of the v1.0 passes. (b) Bytes read per token are deterministic (×0.9671); if only their amount counted, v1.2 would sit on the dashed curve, at the hollow diamond. [meas.] Data: `mesures/genome/construction/codon/j1-s2-passes.csv`, `j1-s2-verdicts.json`.

$r(B/A) = 2.889/2.004 = 1.442$, $\sigma = 5.8\,\%$ (`genome/JUGEMENT-construction.md` § 6.2).

4.5 Where the speed comes from: the read pattern, on this disk

3.3 % fewer bytes cannot return 45 % (Figure 4b); the time spent reading per token falls by a third (Table 3), at almost the same number of bytes. In the same session s2, the disk stage alone (`troncons disque`, no compute) reads the original v1.0 at 1.742 and 1.741 GB/s and the v1.2 at 2.254 and 2.391 [meas.]. A third file, the v1.0 recoded on 17 October, bit-equal to the original, reads at 1.356 and 1.331 GB/s: it carries 267 473 extents against 65, and its fragmentation costs it ×1.26 on the engine's read path (`genome/JUGEMENT-placement.md` § 5.9).

From the two v1.0 files of equal content, the construction step first concluded that the gap was a matter of physical layout, and its judge agreed:

> ~~This gain is not attributable to the format. […] The share of the format is bounded by the ratio of the bytes read: at most +3.4 % with the disk limiting.~~

(Translated from the French original.) This was struck through on 27 October (`genome/06-codon-v1-2.md` § 8), after a second judge, of the disk placement, read *one and the same file* with the request boundaries of

Table 4: The crossed pattern: gpt-oss-20b, disk stage alone, the same file read with the request boundaries of the other format, GB/s (judge of the disk placement, session of 26 October) [meas.].

Case	Four passes	Median
v1.0 file, native pattern	1.792; 1.793; 1.813; 1.809	1.801
v1.0 file, pattern of v1.2	2.509; 2.493; 2.483; 2.564	2.501
v1.2 file, native pattern	2.511; 2.487; 2.556; 2.545	2.528
v1.2 file, pattern of v1.0	1.811; 1.829; 1.805; 1.774	1.808

Source: `mesures/disque/placement/juge/seance/Jb3-*`; `genome/JUGEMENT-placement.md` § 5.3.

the other format, without rewriting anything (Table 4, Figure 5a). At the thresholds declared beforehand (self-noise $\leq 3.2\,\%$, $S_j = 11.8\,\%$), gpt-oss-20b's v1.0 file read with the pattern of its v1.2 file runs ×1.39 faster (1.388 from the unrounded medians, 1.389 in the judge's text from the rounded ones), and the v1.2 file read with the v1.0 pattern runs ×0.715: *slowness follows the pattern, not the file.* Two further experiments bound the variable. Requests shortened by 3.5 % make the v1.0 file fast (×1.421) and requests lengthened by 3.5 % make the v1.2 file slow (×0.713); but cutting each v1.0 chunk into two requests of 6 MB, the same bytes, slows it further (×0.851 with eight reads in flight, ×0.882 with four). Length alone is not the variable. Descriptively, slow requests end exactly at the end of a v1.0 chunk or overrun a v1.2 chunk end by 3.5 %; fast ones stop 3.5 % before a v1.0 chunk end, or end exactly at a v1.2 chunk end; this is not a mechanism, and the exact variable is not isolated (`genome/JUGEMENT-placement.md` §§ 5.3, 5.6, 5.7).

On the 120b, the judge's three-file session splits the end-to-end ratio into a ceiling for the bytes and a remainder [calc.]:

$$r(B/A) = 1.442 = \underbrace{1/0.9671}_{\text{bytes: } \leq 1.034} \times \underbrace{1.394}_{\text{pattern: remainder}},$$

with the disk stage alone, in the same session, reading B 1.37 times faster than A (Figure 5b). The remainder is a lower bound on the effect of the pattern. **On this laptop, the v1.2 format makes the engine about 45 % faster on gpt-oss-120b, through the pattern of its read requests and not through its 3.3 % of bytes; this gain does not carry over to another disk without being measured there.** The amplification from the disk stage (×1.37) to the engine (×1.44) remains a hypothesis: the GPU, which waits more on the disk, may run at lower clocks; its copy and decoding times per token differ by far more than the bytes (Table 3).

5 What did not work

Volume III tried more than it kept. Each attempt below had its threshold committed before the reading, and each verdict was redone by a judge with their own code (Table 5).

A guard on the router margin. Volume II had found that routing flips carry most of the divergence between two close computations: imposing the reference routing removes 88 % of it. The router of a layer scores 32 experts and keeps four; its input space is cut into polyhedral regions, and the margin between the 4th and 5th scores measures the distance to a face [29]. The idea was a guard that recomputes exactly only the cells of small margin. Declared threshold: catch at least 80 % of the flips while flagging at most 10 % of the cells. Measured on gpt-oss-20b: 2 905 flips caught out of 5 087 (57.1 %) for 12 264 cells flagged out of 122 640 (10 %); catching 80 % would require flagging 26.6 % of the cells. The guard falls. Its judge traces the failure to two things: the layers are not alike (layers 8 to 23 catch only 38 to 60 % of their flips), and the secondary flips of cascades come at margins that are still wide; the strict vertical cascade, one flip bringing another in the next layer, explains only a quarter of the flips (`genome/pistes/tropicale/JUGEMENT.md` §§ 4.2, 12).

A power law in the spectra. The hope was that weight matrices might be described by few parameters, as a fractal is. The test fits a power law to the singular values of 48 matrices of gpt-oss-20b drawn before

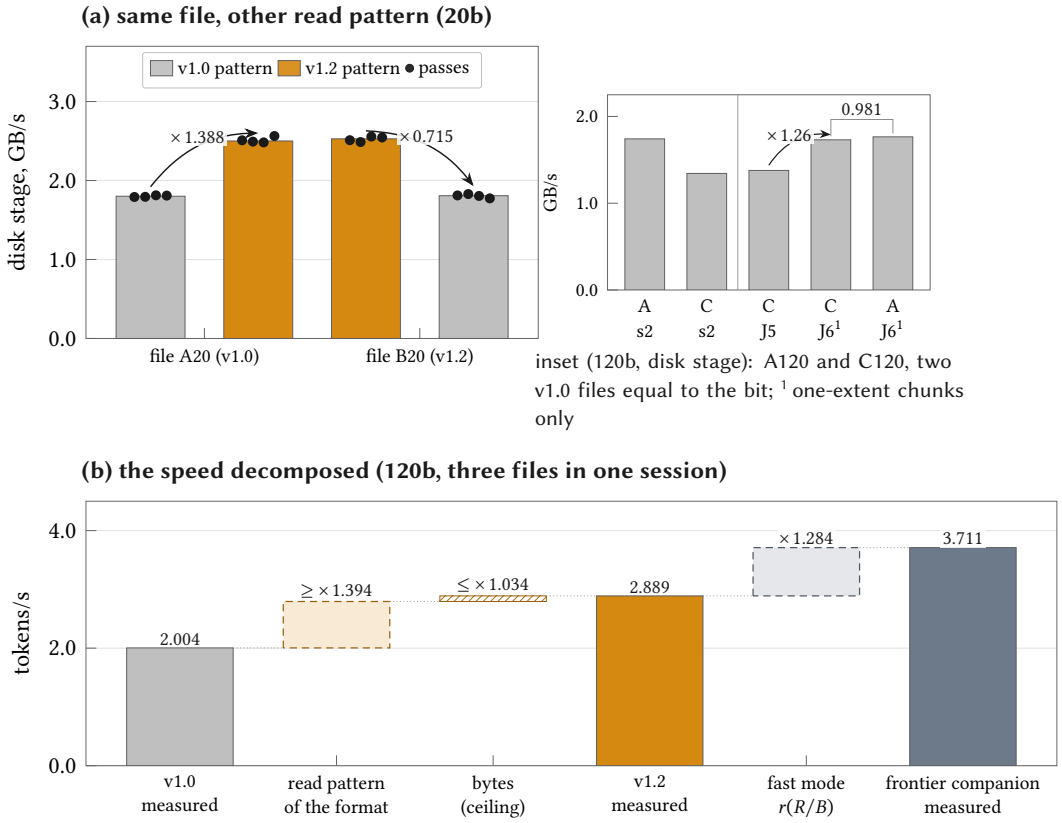


Figure 5: The gain comes from the read pattern that the v1.2 format imposes, on this disk. (a) gpt-oss-20b, disk stage alone: the same file read with the request boundaries of the other format takes the speed of that format (bars: medians; dots: the four passes; colour: the read pattern; ×1.388 is computed from unrounded medians). Inset, gpt-oss-120b: A120 and C120 have the same content to the bit; C120 is slower because of its 267 473 extents (against 65); on its one-extent chunks it reads ×1.26 faster than on chunks drawn at random, and like A120 on the same chunks (0.981). (b) The judge's three-file session on the 120b: $r(B/A) = 1.442$ splits into the bytes (hatched, a ceiling) and the pattern (a remainder, hence a lower bound); the lossy fast companion adds ×1.284 against v1.2. [meas.] Data: `mesures/disque/placement/juge/seance/`, `mesures/genome/construction/codon/disque-s2.json`, `mesures/genome/construction/juge/figures/`.

looking, by the method of Clauset et al. [31]. Declared threshold: at least two thirds of the sample, 32 matrices. Result: 0 out of 48 under the main criterion, 9 out of 48 "compatible" under a criterion amended before the reading. The judge added a theorem: the main criterion could never be met, at any size, because the power law is the boundary of the truncated log-normal family. What holds, and was judged: all 48 matrices have more tail than their permuted version (48 of 48). That this tail sits where a few giant rows are, and that a per-row scale already captures it, comes from an exploratory measurement, neither declared nor judged (`genome/pistes/spectres/JUGEMENT.md`; `genome/pistes/spectres/spectres.md` § 4.3).

Six leads borrowed from other disciplines. Forty-five candidates from biology, physics, mathematics, engineering, human systems and the Earth sciences were sorted into six leads for the expert cache (two-time-scale memory, an exclusive RAM tier, cache-conditioned routing, prefetching with calibrated confidence, annealing, fidelity decided per call). None holds clearly under its judge, each against thresholds declared for it (Volume III, chapter 6). Five stay open, each for a stated reason (among them traces too short, an unmeasured overlap, an unvalidated simulator, synthetic weights). One is at the limit: a router that favours experts already in VRAM removes about 10 % of the misses for a KL divergence of 0.0314 against the same engine without the bias, at the edge of the declared tolerance. A side result is firm: the inclusive RAM tier of the engine adds nothing while it is no larger than the VRAM cache (`genome/mimetisme/`).

Table 5: Negative and open results, each against its declared threshold.

Idea	Declared threshold	Measured, and verdict
router-margin guard	$\geq 80\,\%$ of flips for $\leq 10\,\%$ of cells	57.1 % for 10 %; 80 % costs 26.6 %: **falls**
power law of spectra	≥ 32 of 48 matrices	0 of 48 (9 under the amended criterion): **cannot be written**
cache-conditioned routing	KL to the unbiased engine ≤ 0.038 at $-10\,\%$ misses	0.0314, upper end of the window interval 0.0384: **at the limit**
five other cache leads	declared per lead (Volume III, chapter 6)	**open**, all five
fast mode, generation	$r(R/B) \geq 1.769$ (120b) and ≥ 1.227 (20b): announced gain $\times(1-2\sigma)$	×1.284 and ×0.942 against v1.2: **does not hold**

Source: `genome/pistes/tropicale/seuil-declare.md`, `JUGEMENT.md`; `genome/pistes/spectres/seuil.md`, `JUGEMENT.md`; `genome/mimetisme/P3/resultat-routeur-ligne.md`; `genome/JUGEMENT-construction.md` § 6; Volume III, chapter 13.

The lossy fast mode, as a speed lever. A lossy "synonymous mutation" of the 20b experts wins in quality against the lossy frontier (paired difference $\Delta\mathrm{KL} = -0.1565 \pm 0.0334$ on fresh windows, $4.7\,\sigma$). But the announced gains of the fast mode, ×2.00 on the 120b and ×1.24 on the 20b, compared new files with the old v1.0 file (on the 120b the fast companion was the lossy frontier: the mutation of the 120b was never written); against v1.2, which the disk stage reads at the same speed, they become ×1.284 and ×0.942 [meas.] (`genome/JUGEMENT-construction.md` § 6). The round's generative ideas (a seed with coefficients, shared motifs, duplicated experts) fell earlier on bits: duplication returns 0.0034 bit per weight, of which 0.0016 by chance (Volume III, chapters 3 and 4).

6 Limitations: what might be wrong

- **One machine, one disk, measurements not replicated elsewhere.** Every speed number holds for this NVMe, behind Intel RST VMD, under BitLocker, on given days. The ×1.45 of v1.2 rests on the read pattern on this disk; the exact variable of the pattern is not isolated, and its mechanism (storage stack, controller, data folded out of the SLC cache) is not observable from here. On another disk the gain may be zero, or larger; the only portable speed bound is the +3.4 % of the bytes.
- **The attribution rests on the 20b at the disk stage.** The crossed pattern was measured on gpt-oss-20b, disk stage alone; on the 120b, the three-file session gives only a lower bound on the pattern's effect. The crossed pattern was measured four days after the judge's main session and after a re-optimisation of the disk; the B/A ratio did not move (1.398 and 1.362 before, 1.404 and 1.406 after), which reassures without proving.
- **One model family, born in MXFP4.** The gain is twice the share of −0 codes. A model quantised by a tool that always writes +0 has nothing to gain (v1.2 then costs zero bytes); a model where the sign of a zero is predictable would gain less. The 180 tensors say that neither is the case for gpt-oss.
- **The proof covers the kernel of today.** Proposition 4 assumes finite inputs and is proved on the GEMV of commit `4473050`. Commit `0be464a`, today's kernel, made the value reader a template parameter and keeps exactly the same `fmaf` in the exact mode; it is the version that was in the tree when the 8 640-row control was committed (`a50c380`). A future kernel that read codes otherwise could break bit equality. The f64 reference's corollary needs a no-underflow hypothesis, measured on five vectors, not proved for every input.
- **Equal cache slots isolate the format, not the product.** With the default number of slots, the shorter largest chunk frees 4 472 832 bytes of VRAM, less than one slot (13 254 144 bytes); the default number probably does not change, but this is not measured. About 2.0 MB of the memory difference is unexplained.
- **Samples and text.** The negative results rest on samples (12 reference tensors, 48 matrices, 32 or 768 experts) and on one corpus (wikitext); the guard on gpt-oss-20b only.

- **The judges share conventions.** The judge of the construction wrote their own decoder and KL code, but used the same engine and the same KL convention; a defect common to both would not be seen. A judge was wrong once (“the state of the files”), and was struck through; nothing guarantees that no other reading will come.

7 Conclusion

MXFP4 gives zero two codes. In gpt-oss the choice between them is a fair coin on which no computation of the engines considered depends, and its price is exactly what Corollary 2 says: 0.12562 bit per expert weight on gpt-oss-120b, 1.80 GB on disk. The fold enters a format through its coding tables alone, with a proof that the kernel returns the same bits and a measurement that the logits do. On the laptop where all of this was measured, the new format also made the engine about 45 % faster, for a reason that belongs to the format’s read pattern on this disk and not to its bytes; that number should be measured again before it is believed elsewhere. The other ideas, some of them beautiful, fell at thresholds declared before they were read. Volume II tells how a 63.4 GB model is served from a 384 MB host working set and a GPU footprint set by the card, its logits equal to the bit to those of the raw file; Volume III tells the whole search, its round, its construction and its leads.

Availability

The troncons engine (Rust workspace and CUDA kernels), the tools, the committed measurement logs from which every number of this paper is drawn, and the three volumes [2–4], each in English and French, are released as a technical report at https://github.com/stonearie1491-sudo/troncons. Paths quoted in this paper are relative to the root of that repository. The figures are regenerated from committed data by committed scripts (`memoire/outils/tome3_codon.py`, `memoire/article/figures/`).

References

[1] OpenAI. gpt-oss-120b & gpt-oss-20b model card. arXiv:2508.10925, 2025. URL https://arxiv.org/abs/2508.10925. Accessed 2026-10-15.

[2] Stone. troncons: running a model far larger than memory. Research thesis, Parts I and II: the measured foundations (Volume I). Technical report, 2026. English and French versions; https://github.com/stonearie1491-sudo/troncons.

[3] Stone. troncons, Volume II: the system. Technical report, 2026. English and French versions; https://github.com/stonearie1491-sudo/troncons.

[4] Stone. troncons, Volume III: the genome of the weights. Technical report, 2026. English and French versions; https://github.com/stonearie1491-sudo/troncons.

[5] F. H. C. Crick. The origin of the genetic code. *Journal of Molecular Biology*, 38(3):367–379, 1968. doi: 10.1016/0022-2836(68)90392-6.

[6] Ying Sheng et al. FlexGen: High-throughput generative inference of large language models with a single GPU. arXiv:2303.06865 (ICML 2023), 2023. URL https://arxiv.org/abs/2303.06865. Accessed 2026-10-15.

[7] Keivan Alizadeh et al. LLM in a flash: Efficient large language model inference with limited memory. arXiv:2312.11514 (ACL 2024), 2023. URL https://arxiv.org/abs/2312.11514. Accessed 2026-10-15.

[8] Yixin Song et al. PowerInfer: Fast large language model serving with a consumer-grade GPU. arXiv:2312.12456 (SOSP 2024), 2023. URL https://arxiv.org/abs/2312.12456. Accessed 2026-10-15.

[9] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. arXiv:2312.17238, 2023. URL https://arxiv.org/abs/2312.17238. Accessed 2026-10-15.

[10] Leyang Xue et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. arXiv:2401.14361, 2024. URL https://arxiv.org/abs/2401.14361. Accessed 2026-10-15.

[11] Keisuke Kamahori et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. arXiv:2402.07033 (ICLR 2025), 2024. URL https://arxiv.org/abs/2402.07033. Accessed 2026-10-15.

[12] The ggml authors. llama.cpp, version b11515, 2026. URL https://github.com/ggml-org/llama.cpp/releases/tag/b11515. commit 3d65c90d; MIT; Accessed 2026-10-15.

[13] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966. doi: 10.1147/sj.52.0078.

[14] Moshik Hershcovitch, Andrew Wood, Leshem Choshen, Guy Girmonsky, Roy Leibovitz, et al. ZipNN: Lossless compression for AI models. arXiv:2411.05239, 2024. URL https://arxiv.org/abs/2411.05239. Accessed 2026-10-15.

[15] Tianyi Zhang, Mohsen Hariri, Shaochen Zhong, Vipin Chaudhary, Yang Sui, et al. 70% size, 100% accuracy: Lossless LLM compression for efficient GPU inference via dynamic-length float (DFloat11). arXiv:2504.11651 (NeurIPS 2025), 2025. URL https://arxiv.org/abs/2504.11651. Accessed 2026-10-15.

[16] Patrick Yubeaton, Tareq Mahmoud, Shehab Naga, Pooria Taheri, Tianhua Xia, et al. Huff-LLM: End-to-end lossless compression for efficient LLM inference. arXiv:2502.00922, 2025. URL https://arxiv.org/abs/2502.00922. Accessed 2026-10-15.

[17] Elias Frantar and Dan Alistarh. QMoE: Practical sub-1-bit compression of trillion-parameter models. arXiv:2310.16795, 2023. URL https://arxiv.org/abs/2310.16795. Accessed 2026-10-16.

[18] Open Compute Project. OCP microscaling formats (MX) specification, version 1.0, 2023. URL https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf. Accessed 2026-10-15.

[19] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, et al. Microscaling data formats for deep learning. arXiv:2310.10537, 2023. URL https://arxiv.org/abs/2310.10537. Accessed 2026-10-16.

[20] IEEE. IEEE standard for floating-point arithmetic (IEEE std 754-2019), 2019.

[21] Yuzong Chen, Ahmed F. AbouElhamayed, Xilai Dai, Yang Wang, Marta Andronic, George A. Constantinides, and Mohamed S. Abdelfattah. BitMoD: Bit-serial mixture-of-datatype LLM acceleration. arXiv:2411.11745 (v1, 2024-11-18; v2, 2025-04-26; HPCA 2025), 2024. URL https://arxiv.org/abs/2411.11745. Accessed 2026-10-29.

[22] Yun-Chen Lo, Gu-Yeon Wei, and David Brooks. Nanoscaling floating-point (NxFP): NanoMantissa, adaptive microexponents, and code recycling for direct-cast compression of large language models. arXiv:2412.19821 (v1, 2024-12-15), 2024. URL https://arxiv.org/abs/2412.19821. Accessed 2026-10-29.

[23] Yuzong Chen, Xilai Dai, Jake Hyun, Chi-Chih Chang, Wonsuk Jang, Yuheng Wu, Thierry Tambe, Jae-sun Seo, and Mohamed S. Abdelfattah. RaZeR: Pushing the limits of NVFP4 quantization with redundant zero remapping. arXiv:2501.04052 (v1, 2025-01-06; v3, 2026-01-31), 2025. URL https://arxiv.org/abs/2501.04052. Accessed 2026-10-29.

[24] Anat Heilper and Doron Singer. Lossless compression of neural network components: Weights, checkpoints, and K/V caches in low-precision formats. arXiv:2508.19263 (v1, 2025-08-20), 2025. URL https://arxiv.org/abs/2508.19263. Accessed 2026-10-29.

[25] Jarek Duda. Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding. arXiv:1311.2540, 2013. URL https://arxiv.org/abs/1311.2540. Accessed 2026-10-15.

[26] Fabian Giesen. Interleaved entropy coders. arXiv:1402.3392, 2014. URL https://arxiv.org/abs/1402.3392. Accessed 2026-10-15.

[27] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, 2nd edition, 2006. doi: 10.1002/047174882X.

[28] S. Kullback and R. A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. doi: 10.1214/aoms/1177729694.

[29] Liwen Zhang, Gregory Naitzat, and Lek-Heng Lim. Tropical geometry of deep neural networks. arXiv:1805.07091, 2018. URL https://arxiv.org/abs/1805.07091. Accessed 2026-10-17.

[30] Charles H. Martin and Michael W. Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. arXiv:1810.01075, 2018. URL https://arxiv.org/abs/1810.01075. Accessed 2026-10-17.

[31] Aaron Clauset, Cosma Rohilla Shalizi, and M. E. J. Newman. Power-law distributions in empirical data. *SIAM Review*, 51(4): 661–703, 2009. doi: 10.1137/070710111.

[32] Microsoft. I/O completion ports, 2025. URL https://learn.microsoft.com/en-us/windows/win32/fileio/i-o-completion-ports. Accessed 2026-10-15.

[33] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. arXiv:2309.00071, 2023. URL https://arxiv.org/abs/2309.00071. Accessed 2026-10-15.